

switch-case Yapısı ve Uygulamaları

Görsel Programlama - Hafta 10

C# Karar Yapıları III: switch-case Yapısı ve Uygulamaları

Bu sunum, C# programlama dilindeki switch-case karar yapısını, çalışma prensiplerini, if-else yapılarıyla karşılaştırmasını ve Windows Forms ortamında iki ana uygulama projesi üzerinden pratik kullanımını detaylı olarak incelemektedir.

Hedef: Öğrencilerin, karmaşık karar mekanizmalarını temiz ve verimli switch-case yapıları ile yönetme becerisini kazanması.



3. Hafta: Karar Yapıları: switch-case

Temel Konsept: Bir değişkenin değerini birden çok sabit değerle karşılaştırmak için kullanılan yapı.

Anahtar Kelimeler: switch, case, break, default.

Karşılaştırma: switch-case, genellikle if-else if merdivenlerine göre daha okunaklı bir alternatif sunar.

İşlev Kısıtlaması: switch-case yalnızca eşitlik (==) kontrolü yapabilir.

Neden Karar Yapılarına İhtiyaç Duyarız?

Yazılım, dışarıdan gelen girdilere (kullanıcı eylemi, veri tabanı sonucu, sensör okuması) göre farklı yollar izlemelidir.

Karar yapıları (seçim yapıları), koşullu yürütme prensibini sağlar.

Bu yapılar, kodun sadece doğrusal ilerlemesini engelleyerek uygulamaya 'zeka' katar.



if-else if Merdivenleri

if-else if yapısı, geniş aralıklar ve karmaşık boolean ifadeler (AND, OR) için idealdir.

Ancak, aynı değişkeni 10 veya daha fazla sabit değerle karşılaştırmak gerektiğinde (örneğin menü seçimleri), kod okunabilirliği hızla düşer. Bu durum, switch-case yapısının devreye girdiği temel noktadır.



switch-case'in Tanımı ve Amacı

switch-case, tek bir değişkenin olası tüm sabit değerlerini listeleyerek, ilgili kodu hızlıca çalıştırmayı sağlayan bir kontrol yapısıdır. Programcının niyeti (hangi değer için hangi aksiyon) if-else'e göre daha net görünür. Özellikle enumeration (enum) tipleriyle çalışırken en çok tercih edilen yapıdır.



switch-case ve if-else Karşılaştırması 1/3

switch-case, KESİNLİKLE sadece eşitlik ($=$) kontrolü yapar.

Örnek: case 1, case 'A', case 'Hata'.

if-else, aralık kontrolü yapabilir ($>$, $<$, $>=$, $<=$).

Örnek: if (puan $>$ 90) veya if (yas $<$ 18).

Bu işlevsel fark, hangi yapının seçileceğini belirleyen en kritik faktördür.

switch-case ve if-else Karşılaştırması 2/3

Çok sayıda sabit değere karşı kontrol yapıldığında, modern C# derleyicileri switch-case yapısını bazen JUMP TABLE (atlama tablosu) kullanarak optimize edebilir.

Bu, özellikle tam sayı veya enum gibi sıralı değerlerde performansı artırabilir. if-else if zincirleri ise her bir koşulu sırayla değerlendirmek zorundadır.



switch-case ve if-else Karşılaştırması 3/3

Okunabilirlik: switch-case, bir değişkenin olası durumlarını dikey bir liste halinde gösterdiği için, yeni bir durum eklemek veya mevcut bir durumu değiştirmek if-else if zincirine göre daha az hata riski taşır.
switch'in net yapısı, kod incelemesini (code review) basitleştirir.



C# switch Söz Dizimi (Syntax)

```
switch (kontrol edilen değişken)
{
    case sabit_değer_1:
        // Kod bloğu 1
        break;
    case sabit_değer_2:
        // Kod bloğu 2
        break;
    // ... Diğer case'ler ...
    default:
        // Eşleşme yoksa çalışır
        break;
}
```

switch Anahtar Kelimesi

switch anahtar kelimesi, parantez içinde, karşılaştırılması istenen değişkeni veya ifadeyi barındırır.

Bu ifade, derleme zamanında değeri belirlenebilecek bir sabit değer tipinde olmalıdır (int, string, enum vb.).

Örneğin, 'switch (gunAdı)' veya 'switch (kullaniciSecimi)' gibi.



case Anahtar Kelimesi

case, switch ifadesinin beklediği spesifik sabit değeri tanımlar.
Her case bloğu, switch ifadesiyle tam olarak eşleştğinde çalışır.
Örnek: 'case 1:' veya 'case 'Pazartesi':'.
Aynı switch bloğunda iki case etiketi aynı değere sahip olamaz.

break Anahtar Kelimesi: Zorunlu Çıkış

C# dilinde, bir case bloğu içindeki kod çalıştırıldıktan sonra switch bloğundan çıkış yapmak için break anahtar kelimesi kullanılması ZORUNLUDUR. Eğer break kullanılmazsa, derleyici 'control cannot fall through' hatası verir. Bu kural, C/C++ gibi dillerde istenmeyen kod yürütme (fall-through) durumlarını engeller ve güvenliği artırır.

default Anahtar Kelimesi

default, switch ifadesinin değeri yukarıdaki case etiketlerinden hiçbirisiyle eşleşmediğinde çalıştırılacak varsayılan kod bloğunu belirtir.

default bloğu zorunlu değildir, ancak kullanıcı hatalarını veya beklenmeyen durumları ele almak için çok önemlidir (Hata yönetimi).

default bloğu, switch bloğunun herhangi bir yerine yerleştirilebilir, ancak genellikle en sonda kullanılır.

Case Gruplaması (Fall-Through)

C#'ta birden fazla case etiketinin aynı kod bloğunu çalıştırmasına izin veren tek mekanizma 'case gruplaması'dır.

Bu, break komutu olmadan ardışık case etiketlerini listelemekle sağlanır.

Örnek: 'case 12: case 1: case 2: Label.Text = 'Kış'; break;'.
'

Bu yöntem, mantıksal olarak aynı sonucu veren birden fazla girdi için kodu tekrar yazmaktan kaçınmayı sağlar.

switch Bloğundan Çıkış Yöntemleri

Her case bloğunun sonunda sadece break kullanmak zorunlu değildir.

Dört temel çıkış yöntemi vardır:

1. break: switch bloğundan çıkar.
2. return: Metottan tamamen çıkar (değer döndürerek veya döndürmeyerek).
3. throw: Bir istisna (exception) fırlatır.
4. goto case: Akışı başka bir case etiketine yönlendirir (nadiren kullanılır ve genellikle önerilmez).

switch Yapısında Kapsam (Scope)

switch bloğu içinde tanımlanan yerel değişkenler, tüm switch bloğunda kapsam içinde kalır.

Bu, aynı değişken adının farklı case bloklarında yeniden tanımlanamayacağı anlamına gelir.

Karmaşık case blokları için blok (süslü parantez) kullanmak, değişken kapsamını daraltmaya yardımcı olabilir, ancak standart C# switch yapısında genellikle gerekli değildir.

Geleneksel switch Kısıtlamaları

C# 7.0 öncesinde switch yapısının temel kısıtlamaları şunlardı:

1. Sadece sabit değerler (const) kontrol edilebilirdi.
2. Aralık kontrolleri (> <) yapılamazdı.
3. Sadece primitive tipler (int, string, char, bool) ve enum tipleri destekleniyordu.

Bu kısıtlamalar, C# 7.0 ve sonrasındaki 'Pattern Matching' özellikleri ile büyük ölçüde ortadan kalkmıştır.

switch Pattern Matching (C# 7.0+)

Pattern matching, switch yapısının işlevselliğini büyük ölçüde artırdı.

Artık switch, yalnızca değerleri değil, aynı zamanda değişkenin çalışma zamanındaki tipini de kontrol edebilir.

'case int i:' kalıbı, değişkenin int tipinde olup olmadığını kontrol eder ve değeri i değişkenine atar.

'when' koşulu ile case'lere mantıksal filtreler eklenebilir (örn: 'case int i when i > 10:').

switch Expressions (C# 8.0+)

switch ifadeleri, geleneksel switch-case yapısından farklı olarak bir ifade (expression) olarak çalışır ve bir değer döndürür.

Bu yapı, daha az boilerplate (tekrar eden kod) ile tek satırda sonuç atamayı sağlar.

Söz dizimi, ' $\Rightarrow$ ' (lambda) operatörü ve virgülle ayrılmış durumları kullanır. Genellikle fonksiyonel programlama tarzına daha yakındır.

switch Expressions Söz Dizimi

Örnek Kullanım:

```
'string gunTuru = day switch'  
{  
  ' 1 => 'Hafta İçi',  
  ' 6 or 7 => 'Hafta Sonu',  
  ' _ => 'Geçersiz'  
};
```

Not: Bu ifadede geleneksel break ve case anahtar kelimeleri kullanılmaz.

'_' (underscore) varsayılan durumu (default) temsil eder.

Trafik Lambası Projesi: Kontrol Yapıları

Windows Forms (veya benzeri GUI ortamları) projelerimiz için temel görsel kontroller gereklidir.

Kullanılacak Ana Kontroller:

1. TextBox: Kullanıcı girdisi almak için (Ay Numarası Projesi).
2. Button: Olayları tetiklemek için (Hesapla, Başlat/Durdur).
3. Label: Sonucu veya durumu görüntülemek için.

Trafik Lambası Projesi: Görsel Kontroller

4. Panel: Trafik ışıklarını (Kırmızı, Sarı, Yeşil) görselleştirmek için. Panelin 'BackColor' özelliği değiştirilerek ışık durumu simüle edilir.
5. Timer: Belirli zaman aralıklarıyla tekrarlanan olayları tetiklemek için (Trafik akışını simüle etmek).

Proje 1: Ay Numarasına Göre Mevsim

Amaç: Kullanıcıdan alınan 1-12 arasındaki ay numarasına göre o ayın hangi mevsime ait olduğunu belirlemek ve sonucu göstermektir.

Bu proje, C#'ta izin verilen case gruplaması (ardışık case etiketleri) kullanımını mükemmel şekilde gösterir.



Proje 1: Arayüz Tasarımı

1. Ay Numarası Girişi: Tek bir TextBox kontrolü.
 2. Hesaplama Düğmesi: Tek bir Button kontrolü.
 3. Sonuç Görüntüleme: Tek bir Label kontrolü (Mevsim sonucu).
- Kullanıcıdan alınan girişin tamsayıya dönüştürülmesi (parsing) gerektiği unutulmamalıdır.

Proje 1: Kod Akışı (Button Click)

Butona tıklandığında (Button Click Olayı):

1. TextBox'tan metin değeri alınır.
2. Bu değer, 'int.Parse()' veya 'int.TryParse()' gibi bir yöntemle tamsayıya (ay numarası) dönüştürülür.
3. Dönüştürülen ay numarası, switch yapısına gönderilir: 'switch (ay)'.

Kış Mevsimi Gruplaması

Kış mevsimi, Aralık (12), Ocak (1) ve Şubat (2) aylarını kapsar.

switch yapısında bu üç ayı gruplamak için break kullanılmadan ardışık case etiketleri listelenir:

```
'case 12:'
```

```
'case 1:'
```

```
'case 2:'
```

```
'Label.Text = 'Kış';'
```

```
'break;'
```



İlkbahar Mevsimi Gruplaması

İlkbahar ayları (Mart, Nisan, Mayıs) için benzer bir gruplama yapılır:

```
'case 3:'
```

```
'case 4:'
```

```
'case 5:'
```

```
'Label.Text = 'İlkbahar';'
```

```
'break;'
```

Bu teknik, üç farklı girdinin tek bir çıktı ile sonuçlanmasını sağlar.

Yaz ve Sonbahar Gruplamaları

Yaz ayları (Haziran, Temmuz, Ağustos):

'case 6: case 7: case 8: Label.Text = 'Yaz'; break;'

Sonbahar ayları (Eylül, Ekim, Kasım):

'case 9: case 10: case 11: Label.Text = 'Sonbahar'; break;'

Toplamda 12 ayın tümü 4 farklı case bloğu altında toplanmış olur.

Geçersiz Girdi Yönetimi

Kullanıcı 1-12 aralığı dışında bir sayı girdiğinde (örn: 0 veya 15), bu değer hiçbir case etiketiyle eşleşmeyecektir.

Bu durumu ele almak ve kullanıcıya geri bildirim sağlamak için default bloğu kritik öneme sahiptir.

```
'default:'
```

```
'Label.Text = 'Geçersiz Ay Numarası';'
```

```
'break;'
```

Proje 2: Trafik Lambası Simülasyonu

Amaç: switch-case yapısını kullanarak, Kırmızı, Sarı ve Yeşil durumları arasında periyodik olarak geçiş yapan basit bir Durum Makinesi (State Machine) oluşturmak.

switch-case, bir durum değişkeninin (sayac) farklı sabit değerlerine tepki vererek sistemin durumunu yönetir.



Proje 2: Kullanılacak Kontroller

1. Paneller (3 adet): pnlKirmizi, pnlSari, pnlYesil. Başlangıçta hepsi gri olmalıdır.
2. Buton (1 adet): Simülasyonu başlatıp durdurmak için.
3. Timer (1 adet): Simülasyonun ritmini belirler. Örn: Interval = 1000ms (1 saniye).

Timer Bileşeninin Yapılandırılması

Timer kontrolünün en önemli özelliği Interval'dir. Bu değer (milisaniye cinsinden), Tick olayının ne sıklıkla tetikleneceğini belirler.

Projede 1000ms (1 saniye) kullanmak, geçişlerin rahatça gözlemlenmesini sağlar.

Button'a tıklandığında 'Timer.Start()' veya 'Timer.Stop()' komutları verilerek simülasyon başlatılır/durdurulur.



Durum Değişkeninin Tanımlanması

Simülasyonun hangi aşamada olduğunu izlemek için Form seviyesinde (class scope) bir tamsayı değişkeni tutulur:

```
'int sayac = 0;'
```

Bu sayaç, her Tick olayında artırılır ve switch-case yapısının kontrol ettiği ana değer haline gelir.

1: Kırmızı, 2: Sarı, 3: Yeşil durumlarını temsil edecektir.

Timer_Tick Olayı Mantığı

Timer_Tick olayı, Timer'ın Interval süresi dolduğunda otomatik olarak çalışır.

Bu olayın ilk adımı, sayacı bir artırmaktır:

`'sayac++;'`

Ardından, yeni sayaç değerine göre switch-case ile hangi ışığın yanacağına karar verilir: `'switch (sayac)'`.

Durum 1: Kırmızı Işık

Sayaç 1 olduğunda, Kırmızı Işık durumu aktif edilir:

```
'case 1: // Kırmızı'
```

```
'pnlKirmizi.BackColor = Color.Red;'
```

Diğer ışıklar söndürülür (Gri renk):

```
'pnlSari.BackColor = Color.Gray;'
```

```
'pnlYesil.BackColor = Color.Gray;'
```

```
'break;'
```

Durum 2: Sarı Işık

Sayaç 2 olduğunda, Sarı Işık durumu aktif edilir (Kırmızıdan Yeşile geçiş öncesi):

```
'case 2: // Sarı'  
'pnlKirmizi.BackColor = Color.Gray;'  
'pnlSari.BackColor = Color.Orange;'  
'pnlYesil.BackColor = Color.Gray;'  
'break;'
```

Durum 3: Yeşil Işık

Sayaç 3 olduğunda, Yeşil Işık durumu aktif edilir:

```
'case 3: // Yeşil'  
'pnlKirmizi.BackColor = Color.Gray;  
'pnlSari.BackColor = Color.Gray;  
'pnlYesil.BackColor = Color.Green;  
'break;
```

Döngünün Kapanması ve Sıfırlama

Trafik lambası döngüsel olarak çalışmalıdır (Kırmızı -> Sarı -> Yeşil -> Kırmızı...). Yeşil Işık (case 3) tamamlandığında, sayacın 0'a geri dönmesi gerekir:
'case 3:' bloğunun içinde:
'sayac = 0;'
Bir sonraki Timer_Tick olayında, sayaç tekrar 1 olacak ve Kırmızı Işık durumu tetiklenecektir.



Hata Yönetimi ve default Case

Trafik lambası projesinde sayacın değeri teorik olarak sadece 1, 2 veya 3 olmalıdır.

Ancak programlama hatalarına karşı koruma sağlamak için default bloğu eklenebilir:

```
'default: sayac = 0; // Beklenmeyen durumda döngüyü sıfırla'
```

Bu, uygulamanın beklenmedik bir sayaç değerinde çökmesini önler.



Durum Makineleri (State Machines)

Trafik lambası projesi, switch-case'in basit bir durum makinesi uygulamasıdır. Durum makinesi, sistemin farklı modlar (durumlar) arasında kontrollü bir şekilde geçiş yapmasını sağlar. switch-case, mevcut durumu (sayac) kontrol ederek bir sonraki durumu belirlemek için ideal bir yapıdır.



switch-case ve Enum'lar

switch yapısı, tamsayılar yerine 'enum' (sabitler kümesi) tipleriyle kullanıldığında en yüksek okunabilirliği sağlar.

Örnek: 'switch (Durum.Kirmizi)' yerine 'case TrafikDurumu.Kirmizi:'.
Enum kullanımı, kodda sihirli sayıların (magic numbers) önüne geçer ve programın amacını netleştirir.



switch-case: En İyi Uygulamalar (Best Practices)

1. Mümkünse string yerine enum kullanın.
2. default bloğunu her zaman dahil edin (hata yakalama veya varsayılan işlem için).
3. Her case'i break ile sonlandırdığınızdan emin olun (case gruplaması hariç).
4. Eğer case blokları çok uzunsa ve karmaşık mantık içeriyorsa, bu mantığı ayrı bir metoda taşıyın.

Ne Zaman switch, Ne Zaman if-else?

if-else kullanın:

- Aralık kontrolü ($>$ $<$) gerektiğinde.
- Karmaşık boolean koşullar ($\&\&$, $\|\$) kontrol edildiğinde.
- Değişkenin tipi birden fazla kez değişiyorsa (pattern matching öncesi).

Ne Zaman switch, Ne Zaman if-else? (Devam)

switch-case kullanın:

- Tek bir değişkenin değeri birden çok sabit, ayrık değerle karşılaştırıldığında.
- Durum makinesi veya menü seçimleri yönetildiğinde.
- Enum tiplerinin değerleri üzerinde çalışıldığında.

switch İfadelerinin Avantajları

Geleneksel switch yapısının aksine, switch ifadeleri değer döndürdüğü için doğrudan bir değişkene atama yapılabilir.

Bu, kod tekrarını azaltır (break, return, throw yazma zorunluluğu yoktur). switch ifadeleri, pattern matching ile birlikte kullanıldığında, karmaşık tip kontrollerini çok daha temiz bir şekilde yapmayı mümkün kılar.

Örnek Senaryo: Hesap Makinesi

Kullanıcıdan bir işlem sembolü (örn: '+', '-', '*') alındığında, hangi matematiksel operasyonun yapılacağına karar vermek için switch-case idealdir.

'switch (operasyon)'

'case '+': // Toplama metodu'

'case '-': // Çıkarma metodu'

Bu tür menü bazlı kontroller, switch-case'in klasik kullanım alanıdır.

Örnek Senaryo: Hata Kodları

Bir sistemden dönen tamsayı hata kodlarını (Error Codes) kullanıcı dostu mesajlara çevirmek için switch-case kullanılabilir.

```
'switch (errorCode)'
```

```
'case 404: Label.Text = 'Sayfa bulunamadı'; break;'
```

```
'case 500: Label.Text = 'Sunucu hatası'; break;'
```

Bu, kodların ayrık ve yönetilebilir olmasını sağlar.

Veri Tiplerinin Uyumluluğu

Geleneksel switch yapısı, C#'ın başından beri tüm integral tipleri (int, short, byte, long), char, string ve enum tiplerini desteklemiştir.

Önemli Not: double (kayan nokta) tipleri, hassasiyet sorunları nedeniyle genellikle switch kontrolünde kullanılmaz.



Case Gruplaması Detaylı İnceleme (P1)

Ay Numarasına Göre Mevsim projesinde, 'case 12:' bloğundan hemen sonra break olmamasına rağmen derleyici hata vermez.

Bunun nedeni, case 12'nin altında hiçbir yürütülebilir kodun olmamasıdır. C# bu yapıyı özel olarak, birden fazla case'in bir sonraki kod bloğuna atlamasına izin veren bir 'gruplama' mekanizması olarak kabul eder.



TextBox Kontrolü ve Güvenlik

Ay Numarası projesinde, kullanıcı TextBox'a sayı yerine harf girebilir. Bu durumda 'int.Parse()' kullanmak uygulama çökmesine (istisna fırlatmasına) neden olur.

Üniversite düzeyinde, 'int.TryParse()' metodunun kullanılması (daha güvenli dönüşüm) ve default case'de hata mesajı verilmesi beklenir.



Simülasyon Hızı ve Timer Interval

Timer'ın Interval'i, durumlar arası bekleme süresini belirler.

1000ms = 1 saniye bekleme demektir.

Daha gerçekçi bir trafik akışı simülasyonu için Kırmızı ve Yeşil ışık durumlarında Sarı ışığa göre daha uzun aralıklar kullanılmalıdır (örn: Kırmızı=3000ms, Sarı=1000ms, Yeşil=4000ms).



Trafik Lambası (Gelişmiş) State Machine

Daha karmaşık simülasyonlarda, sadece sayaç yerine her durum için ayrı bir Timer veya switch-case yapısının içinde ek bir süre kontrolü gerekebilir. Örneğin, Kırmızı ışık 3 saniye yanar, ardından Sarıya geçmeden önce ek bir bekleme süresi eklenebilir.



Panel Kontrolünün Amacı

Panel kontrolü, sınırlandırılmış bir alanda görsel düzenlemeler yapmaya olanak tanır.

Trafik lambası projesinde, ışıkların görselleştirilmesi için Panel'in 'BackColor' özelliğinin programatik olarak Color.Red, Color.Green ve Color.Gray değerleriyle değiştirilmesi esastır.

Label veya PictureBox yerine Panel kullanımı genellikle daha temiz bir çözüm sunar.



switch Yapısının Mimari Önemi

switch-case, özellikle komut tabanlı sistemlerde (Command Pattern) gelen komutun tipine göre farklı metotları çağıran bir 'ayırma' katmanı görevi görür. Bu, kodun farklı bölümlerinin birbirine bağımlı olmadan çalışmasını sağlayan modüler bir tasarım prensibidir.

C# String switch Özelliği

C# 7.0 öncesi dillerde string tabanlı switch kullanmak bazen yavaş olabilirdi. Ancak C# ve .NET Core, string switch'i optimize ederek hash tablosu benzeri yapılar kullanır.

Bu, string tabanlı komut ve menü seçimlerinde switch-case'i if-else if zincirlerine göre neredeyse her zaman daha iyi bir seçim haline getirir.



Pattern Matching: is ve switch

C# pattern matching, 'is' ifadesiyle de kullanılabilir.

'if (obj is string s)' yapısı, hem tip kontrolü yapar hem de değişkeni otomatik olarak dönüştürür.

switch pattern matching ise bu kontrollerin toplu, daha temiz bir biçimde yapılmasını sağlar.



Null Değerlerin Kontrolü

C# 7.0'dan itibaren, switch-case yapıları içinde 'case null:' etiketiyle null değerleri doğrudan kontrol etmek mümkündür.

Bu, geleneksel olarak if (değişken == null) ile yapılan null kontrolünün switch yapısı içine dahil edilmesini sağlar.



switch İfadelerinde Mükemmel Eşleşme

switch ifadeleri (switch expressions) kullanılırken, eğer kontrol edilen tip bir enum ise, tüm olası enum değerlerini kapsamak zorunludur.

Eğer tüm değerler kapsamazsa, derleyici hata verir veya default ('_') operatörünü kullanmak gerekir.

Bu kural, kodun eksik durumları göz ardı etmesini engeller.



Karar Yapılarının Karmaşıklığı

Yazılım mühendisliğinde, bir metodun test edilebilirliğini ve bakımını ölçmek için döngüsel karmaşıklık metrikleri kullanılır.

Uzun if-else if merdivenleri bu karmaşıklığı artırırken, iyi yapılandırılmış switch-case (özellikle enum'lar ile) karmaşıklığı daha yönetilebilir seviyelerde tutmaya yardımcı olabilir.



switch İfadesi: Tuple Pattern

switch ifadeleri, birden fazla değişkeni aynı anda kontrol eden tuple (demet) kalıplarını destekler.

Örn: '(durum, eylem) switch { (State.Idle, Command.Start) => State.Running, ... }'

Bu, trafik lambası projesi gibi durum makinelerinin daha temiz bir şekilde kodlanmasına olanak tanır.



Proje 1 Kod İncelemesi: Dönüşüm

Gerçek bir uygulamada, TextBox'tan gelen girdi her zaman string'tir.

Tamsayıya dönüştürürken kullanılan kodun, geçersiz format (NaN) istisnalarını yönettiğinden emin olunuz.

'int.TryParse(textBox1.Text, out int ay)' kullanımı, try-catch bloklarına göre daha temiz hata denetimi sağlar.



Proje 2 Kod İncelemesi: Sayaç Mantığı

Trafik lambası simülasyonunda sayacın 'case 3:' sonunda sıfırlanması ('sayac = 0;') döngüselliği sağlar.

Eğer bu sıfırlama unutulursa, sayaç 4 olduğunda default'a düşer veya döngü durur (varsa default bloğuna gider).

Bu, döngüsel durum makineleri için hayati bir adımdır.

Özet ve Temel Çıkarımlar

switch-case, ayırık ve sabit değerli koşullar için en okunaklı ve verimli C# kontrol yapısıdır.

C# 7.0 sonrası Pattern Matching ve switch Expressions ile switch yapısının gücü ve esnekliği önemli ölçüde artmıştır.

Timer ve Panel gibi kontrollerle birleştirildiğinde, switch-case durum makineleri oluşturmak için güçlü bir araçtır.



Sıkça Sorulan Sorular: break Zorunluluğu

C#, C/C++'taki gibi istenmeyen 'fall-through' durumlarının (bir case'in kodunun bitip hemen altındaki case'in kodunun çalışması) önüne geçmek için break'i zorunlu kılar.

Bu tasarım tercihi, hata ayıklama süresini azaltmayı ve kod güvenliğini artırmayı amaçlar.



Sıkça Sorulan Sorular: String switch Hızı

Modern C# switch yapısı, dize karşılaştırmaları için yüksek performanslı algoritmalar kullanır.

Bu nedenle, menü seçenekleri veya komutlar gibi metin tabanlı kontrollerde, performans endişesi olmadan switch kullanılması önerilir.



Gelecek Perspektifi: switch ve Nesne Tabanlı Programlama

Çok sık kullanılması önerilmese de, switch-case bazı durumlarda polimorfizme (çok biçimlilik) alternatif olarak kullanılabilir.

Özellikle yeni başlayanlar için basit tip kontrolleri yaparken temiz bir başlangıç noktası sunar.

Ancak karmaşık nesne hiyerarşilerinde polimorfizm tercih edilmelidir.

1. switch-case yapısını kullanarak bir haftanın gün adını tamsayıya çeviren bir uygulama yazın.
2. Farklı dillerdeki (Java, Python) switch yapılarını ve C#'tan farklarını araştırın.
3. C# 9.0 ile gelen ilişki kalıplarını (relational patterns) ve mantıksal kalıpları (logical patterns) inceleyin.



Kapanış

Karar yapılarının doğru kullanımı, temiz, güvenilir ve sürdürülebilir yazılım geliştirmenin temelidir.

switch-case ve uygulamaları hakkında daha fazla sorunuz varsa lütfen çekinmeyin.



switch-case Yapısı ve Uygulamaları

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

